Zestaw Zadań: React JS — Zaawansowane Podstawy

Część 1: Custom Hooks i useRef

Informacje Ogólne

Materiał źródłowy:	Custom hooks (useLocalStorage, useFetch, useDebounce), useRef — referencje do DOM i wartości mutowalne
Poziom:	2 🟡 Średniozaawansowany
Szacowany czas:	~110 minut (2 zadania)
Typ zestawu:	Powiązane — rozbudowa aplikacji "Menedżer Zadań" z pierwszego zestawu React
Wymagania wstępne:	Ukończony Zestaw Zadań: Podstawy React JS (Zadania 1-6), znajomość HTML, CSS, JavaScript (ES6+)

Dlaczego potrzebujesz więcej niż useState i useEffect?

Podstawowy zestaw React nauczył Cię budowania komponentów, zarządzania stanem przez useState, obsługi efektów ubocznych przez useEffect oraz pracy z formularzami i API. To solidny fundament — ale w miarę rozrastania się aplikacji napotkasz problemy, których podstawowe narzędzia nie rozwiązują elegancko.

Wyobraź sobie Menedżer Zadań z pierwszego zestawu: stan aplikacji żyje w App.jsx i jest przekazywany przez props przez 3-4 poziomy komponentów. Dodajesz dark mode, autoryzację, język interfejsu — i nagle App.jsx ma 15 propsów, a każdy pośredni komponent musi je przekazywać dalej, mimo że sam ich nie używa. To zjawisko nazywa się **prop drilling** i jest jednym z najczęstszych problemów w rosnących aplikacjach React.

Ten zestaw zadań wprowadza narzędzia rozwiązujące te problemy: custom hooks do wyodrębniania reużywalnej logiki, useReducer do zarządzania złożonym stanem, Context API do eliminacji prop drilling, React Router do nawigacji między widokami, oraz techniki optymalizacji wydajności. Po jego ukończeniu będziesz dysponować pełnym zestawem umiejętności wymaganych od junior React developera.

Najczęstsze problemy i pułapki

2. Zapominanie o tablicy zależności w custom hooks

Custom hook to zwykła funkcja — ale jeśli wewnątrz używa useEffect lub useCallback, musi poprawnie deklarować zależności. Brak zależności prowadzi do "stale closures" — zamknięć odwołujących się do nieaktualnych wartości.

```
// ❌ Źle – url się zmienia, ale efekt tego nie widzi
function useFetch(url) {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch(url).then(r => r.json()).then(setData);
  }, []); // Brak url w zależnościach!
  return data;
```

```
}

// ✅ Dobrze – efekt reaguje na zmianę url
function useFetch(url) {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch(url).then(r => r.json()).then(setData);
  }, [url]); // url w tablicy zależności
  return data;
}
```

📌 **Zasada:** Custom hook podlega tym samym regułom hooków co komponent — wywołuj hooki na najwyższym poziomie (nie w ifach ani pętlach), zawsze deklaruj pełną tablicę zależności.

Zadanie 1: Custom Hooks — wyodrębnianie reużywalnej logiki

Poziom trudności:	2 🟡 (Średniozaawansowany)
Szacowany czas:	60 minut
Punktacja:	25 punktów
Typ:	Praktyczne
Powiązanie:	Rozszerza Zadania 3-5 z Podstawy React (refaktoryzacja istniejącej logiki)

Cel dydaktyczny

Opanowanie tworzenia custom hooks do wyodrębniania powtarzalnej logiki z komponentów, co poprawia czytelność kodu i umożliwia jego ponowne użycie w wielu miejscach aplikacji.

Wstęp teoretyczny

Czym jest Custom Hook?

Custom hook to zwykła funkcja JavaScript, której nazwa zaczyna się od `use`, i która wewnętrznie korzysta z innych hooków React. Pozwala "wyciągnąć" powtarzalną logikę z komponentu do osobnej, reużywalnej funkcji.

```
// Custom hook – wyodrębnia logikę localStorage
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}
```

```
// Użycie — identyczny interfejs jak useState
const [tasks, setTasks] = useLocalStorage("tasks", []);
const [theme, setTheme] = useLocalStorage("theme", "light");
```

Zasady custom hooks:

- Nazwa MUSI zaczynać się od `use` (React sprawdza to i wyświetla ostrzeżenia)
- Hook może wywoływać inne hooki (useState, useEffect, inne custom hooks)
- Hook podlega tym samym regułom co inne hooki (nie w ifach, pętlach, zagnieżdżeniach)
- Hook zwraca wartości potrzebne komponentowi (stan, funkcje, obiekty)

Polecenie

Zrefaktoryzuj Menedżer Zadań, wyodrębniając powtarzalną logikę do custom hooks:

Część A — useLocalStorage (8 pkt)

1. Stwórz plik `src/hooks/useLocalStorage.js`
2. Zaimplementuj hook `useLocalStorage(key, initialValue)`:
   - Lazy initial state — odczyt z localStorage tylko przy pierwszym renderze
   - Automatyczny zapis do localStorage przy każdej zmianie wartości
   - Obsługa błędów JSON.parse (try/catch) — gdy dane w localStorage są uszkodzone, użyj initialValue
   - Zwraca `[value, setValue]` — identyczny interfejs jak useState
3. Zastąp istniejącą logikę localStorage w App.jsx nowym hookiem: ```jsx // Przed (App.jsx): const [tasks, setTasks] = useState(() => { const saved = localStorage.getItem('tasks'); return saved ? JSON.parse(saved) : initialTasks; }); useEffect(() => { localStorage.setItem('tasks', JSON.stringify(tasks)); }, [tasks]);

// Po: const [tasks, setTasks] = useLocalStorage('tasks', initialTasks);

```
4. Użyj useLocalStorage również do przechowywania aktywnego filtra i sortowania

#### Część B — useFetch (10 pkt)

1. Stwórz plik `src/hooks/useFetch.js`

2. Zaimplementuj hook `useFetch(url, options)`:
   - Zarządzanie trzema stanami: `data`, `loading`, `error`
   - Automatyczne pobieranie danych przy zmianie `url`
   - Cleanup z AbortController (anulowanie żądania przy odmontowaniu lub zmianie url)
   - Funkcja `refetch()` do ręcznego ponowienia żądania
   - Zwraca obiekt: `{ data, loading, error, refetch }`

3. Zrefaktoryzuj komponent QuoteOfTheDay z Zadania 5 (Podstawy React), aby korzystał z
useFetch:
```

// Przed: const [quote, setQuote] = useState(null); const [loading, setLoading] = useState(true); const [error, setError] = useState(null); useEffect(() => { /* 15 linii logiki fetch */ }, []);

// Po: const { data: quote, loading, error, refetch } = useFetch(QUOTE_API_URL);

```
4. Dodaj opcjonalny parametr `options`:
   - `enabled` (boolean, domyślnie true) — czy automatycznie pobierać dane
   - `transform` (funkcja) — transformacja danych po pobraniu (np. `data =>
data.results`)

#### Część C — useDebounce (7 pkt)

1. Stwórz plik `src/hooks/useDebounce.js`

2. Zaimplementuj hook `useDebounce(value, delay)`:
   - Zwraca zdebouncowaną wartość — aktualizuje się dopiero po `delay` ms od ostatniej
zmiany
   - Cleanup timera przy odmontowaniu i przy każdej zmianie wartości

3. Zastosuj useDebounce do pola wyszukiwania z Zadania 6 (Podstawy React):
```

const [searchInput, setSearchInput] = useState(""); const debouncedSearch = useDebounce(searchInput, 300); // Filtrowanie używa debouncedSearch zamiast searchInput

```
4. Zweryfikuj działanie: przy szybkim wpisywaniu lista nie "mruga" — aktualizuje się
dopiero po zatrzymaniu pisania

### Materiały/Zasoby potrzebne

- Projekt Menedżer Zadań z ukończonego zestawu Podstawy React (Zadania 1-6)
- Dokumentacja React — [Reusing Logic with Custom
Hooks](https://react.dev/learn/reusing-logic-with-custom-hooks)
- Dokumentacja React — [Rules of Hooks](https://react.dev/reference/rules/rules-of-
hooks)

### Kryteria oceny

- useLocalStorage działa poprawnie z odczytem i zapisem (3 pkt)
- Lazy initial state w useLocalStorage (2 pkt)
- Obsługa błędów JSON.parse (1 pkt)
- Zastosowanie useLocalStorage w min. 2 miejscach (2 pkt)
- useFetch zarządza stanami data/loading/error (3 pkt)
- AbortController i cleanup (2 pkt)
- Funkcja refetch (2 pkt)
- Parametr options (enabled, transform) (3 pkt)
- useDebounce działa poprawnie (3 pkt)
- Zastosowanie useDebounce w wyszukiwaniu (2 pkt)
- Czystość kodu i organizacja plików (2 pkt)

### Wskazówki

- Custom hook to po prostu funkcja — nie ma w niej nic magicznego oprócz prefiksu `use`
- Testuj hooki w izolacji: stwórz prosty komponent testowy, który tylko wyświetla wynik
hooka
- useDebounce: kluczem jest `setTimeout` w `useEffect` z cleanup `clearTimeout`
- AbortController: `const controller = new AbortController(); fetch(url, { signal:
controller.signal })` — w cleanup wywołaj `controller.abort()`
- Jeśli hook przyjmuje funkcję jako argument (np. `transform`), pamiętaj o dodaniu jej
do tablicy zależności useEffect lub opakowaniu w useCallback
```

```
### Przykładowa struktura plików
```

task-manager/ ├── src/ │ ├── hooks/ │ │ ├── useLocalStorage.js │ │ ├── useFetch.js │ │ └── useDebounce.js │ ├── components/ │ │ └── ... │ ├── App.jsx │ └── main.jsx └── package.json

```
---


## Zadanie 2: useRef — referencje do DOM i wartości mutowalne

| | |
|---|---|
| **Poziom trudności:** | 2 🟡 (Średniozaawansowany) |
| **Szacowany czas:** | 50 minut |
| **Punktacja:** | 20 punktów |
| **Typ:** | Praktyczne |
| **Powiązanie:** | Rozszerza Zadanie 4 z Podstawy React (formularze i edycja) |

### Cel dydaktyczny

Zrozumienie hooka useRef i jego dwóch głównych zastosowań: dostęp do elementów DOM (focus, scroll, pomiary) oraz przechowywanie mutowalnych wartości między renderami bez wywoływania re-renderu.

### Wstęp teoretyczny

**useRef — dwa oblicza jednego hooka**

useRef tworzy obiekt `{ current: wartość }`, który przetrwa między renderami. W przeciwieństwie do useState, zmiana `.current` NIE powoduje re-renderu.
```

import { useRef, useEffect } from 'react';

// 1. Referencja do elementu DOM function SearchInput() { const inputRef = useRef(null); useEffect(() => { inputRef.current.focus(); // Focus na input po zamontowaniu }, []); return <input ref={inputRef} placeholder="Szukaj..." />; }

// 2. Przechowywanie wartości mutowalnej (bez re-renderu) function Timer() { const intervalRef = useRef(null); const start = () => { intervalRef.current = setInterval(() => console.log("tick"), 1000); }; const stop = () => clearInterval(intervalRef.current); return (/* przyciski start/stop */); }

```
**Kiedy useRef zamiast useState?**
- Potrzebujesz dostępu do elementu DOM (focus, scroll, wymiary)
- Przechowujesz wartość, której zmiana NIE powinna wywoływać re-renderu (timer ID, poprzednia wartość, licznik renderów)
- Potrzebujesz wartości "aktualnej" wewnątrz callbacka lub timera (unikanie stale closures)

### Polecenie
```

```
Rozbuduj Menedżer Zadań wykorzystując useRef w praktycznych scenariuszach:

#### Część A – Auto-focus w formularzach (6 pkt)

1. Dodaj automatyczny focus na pole tytułu po otwarciu formularza dodawania zadania:
```

const titleInputRef = useRef(null); useEffect(() => { titleInputRef.current?.focus(); }, []);

```
2. W trybie edycji zadania (TaskItem) – automatyczny focus na pole edycji po kliknięciu
"Edytuj"

3. Po dodaniu zadania – ponowny focus na pole tytułu (gotowość do dodania kolejnego)

#### Część B – Scroll do nowego zadania (6 pkt)

1. Po dodaniu nowego zadania automatycznie przewiń listę do nowo dodanego elementu:
```

const lastTaskRef = useRef(null); useEffect(() => { if (lastTaskRef.current) { lastTaskRef.current.scrollIntoView({ behavior: 'smooth', block: 'nearest' }); } }, [tasks.length]);

```
2. Dodaj ref do ostatniego elementu listy (hint: callback ref lub warunkowy ref w
.map())

3. Dodaj animację CSS (np. krótkie podświetlenie) nowo dodanego zadania

#### Część C – Poprzednia wartość filtra i licznik renderów (8 pkt)

1. Stwórz custom hook `usePrevious(value)`:
```

function usePrevious(value) { const ref = useRef(); useEffect(() => { ref.current = value; }); return ref.current; }

```
2. Użyj usePrevious do wyświetlenia komunikatu o zmianie filtra:
   - "Zmieniono filtr z «Aktywne» na «Ukończone»"
   - Komunikat znika po 2 sekundach (użyj useRef do przechowywania ID timera)

3. Stwórz hook `useRenderCount()` (debuggowy):
   - Używa useRef do zliczania renderów komponentu
   - Wyświetla liczbę renderów w konsoli (tylko w trybie development)
   - Zastosuj go w 2-3 komponentach, aby zobaczyć, jak często się re-renderują

4. Użyj useRef do przechowywania ID timera dla komunikatów toast

### Materiały/Zasoby potrzebne

- Projekt Menedżer Zadań z Zadania 1 (z custom hooks)
- Dokumentacja React – [useRef](https://react.dev/reference/react/useRef)
```

```
- Dokumentacja React – [Manipulating the DOM with
Refs](https://react.dev/learn/manipulating-the-dom-with-refs)

### Kryteria oceny

- Auto-focus na formularz dodawania (2 pkt)
- Auto-focus w trybie edycji (2 pkt)
- Focus po dodaniu zadania (2 pkt)
- Scroll do nowego zadania (3 pkt)
- Animacja nowo dodanego zadania (3 pkt)
- Hook usePrevious działa poprawnie (3 pkt)
- Komunikat o zmianie filtra z auto-hide (3 pkt)
- Hook useRenderCount (1 pkt)
- Poprawne użycie ref do ID timera (1 pkt)

### Wskazówki

- `element.scrollIntoView({ behavior: 'smooth' })` – płynne przewijanie
- Callback ref: `ref={(el) => { if (isLast) lastTaskRef.current = el; }}`
- `useRef` NIE wywołuje re-renderu – jeśli potrzebujesz re-renderu po zmianie, użyj
useState
- Animacja CSS: dodaj klasę `.just-added` z `@keyframes`, usuń ją po 1-2 sekundach
- Pamiętaj o cleanup timerów: `return () => clearTimeout(timerRef.current)` w useEffect

---



---

## Pomocne zasoby

### Dokumentacja i referencje

- React – Reusing Logic with Custom Hooks: [react.dev/learn/reusing-logic-with-custom-
hooks](https://react.dev/learn/reusing-logic-with-custom-hooks)
- React – useRef:
[react.dev/reference/react/useRef](https://react.dev/reference/react/useRef)
- React – Manipulating the DOM with Refs: [react.dev/learn/manipulating-the-dom-with-
refs](https://react.dev/learn/manipulating-the-dom-with-refs)
- React – Rules of Hooks: [react.dev/reference/rules/rules-of-
hooks](https://react.dev/reference/rules/rules-of-hooks)

### Narzędzia do nauki

- useHooks – kolekcja gotowych custom hooks: [usehooks.com](https://usehooks.com)
- React Patterns: [reactpatterns.com](https://reactpatterns.com)

### Darmowe API do ćwiczeń

- JSONPlaceholder: [jsonplaceholder.typicode.com](https://jsonplaceholder.typicode.com)
- Public APIs lista: [github.com/public-apis/public-apis](https://github.com/public-
apis/public-apis)

---

## Legenda oznaczeń poziomu trudności
```

```
| Poziom | Emoji | Nazwa | Opis |
|--------|-------|-------|------|
| 1 | 🟢 | Podstawowy | Sprawdzenie zrozumienia podstawowych koncepcji |
| 2 | 🟡 | Średniozaawansowany | Zastosowanie wiedzy w typowych sytuacjach |
| 3 | 🟠 | Zaawansowany | Analiza, synteza, rozwiązywanie złożonych problemów |
| 4 | 🔴 | Ekspert | Tworzenie nowej wiedzy, innowacyjne podejścia, projekty
kompleksowe |
```